



Rede solidária (projeto integrador) - Documento definição técnica

Professora responsável: Kadidja Valeria Reginaldo de Oliveira

Curso: ADS

Feito por: Rafael Ramos de Carvalho

Revisado por: Bruna Bonifacio Soares Santos , Renata Teixeira de Jesus e Ana Clara de Almeida Coelho

Descrição do projeto: O Rede Solidária é uma plataforma digital (Web/Mobile) desenvolvida para otimizar a gestão logística e a transparência no ecossistema de doações de itens físicos no Distrito Federal. Diferente de sistemas de crowdfunding financeiro, o foco da plataforma é o inventário de insumos críticos (alimentos, itens de higiene, fraldas geriátricas, insumos hospitalares e vestuário).

O sistema funciona como uma ponte inteligente entre doadores e instituições de pequeno e médio porte, permitindo que a oferta de doações encontre a demanda real de forma balanceada e eficiente.

Data de Criação: 27/05/2026

Versão:1.0

Histórico de versões

Versão	Data	Autor	Descrição alteração
1.0	27/05/2026	Rafael Ramos	Criação do documento

1. Escolha de Linguagem e Frameworks

Para um sistema Web/Mobile que precisa ser desenvolvido de forma ágil e eficiente, a união do ecossistema **JavaScript/TypeScript** é ideal, pois permite que a mesma linguagem seja usada no Front-end, Back-end e Mobile, reduzindo a curva de aprendizado da equipe.

- **Back-end (API):**
 - **Tecnologia: Node.js** com **TypeScript**.
 - **Framework: NestJS** ou **Fastify**. O NestJS é recomendado por trazer uma estrutura organizacional forte e opinativa (arquitetura escalável por padrão), além de excelente suporte nativo a TypeScript.
- **Front-end (Web - Painel das ONGs e Administrador):**
 - **Framework: React.js** ou **Next.js** (com TypeScript).
 - **Biblioteca de UI: Tailwind CSS** (para estilização rápida) combinada com **shadcn/ui** ou **Chakra UI** para componentes acessíveis e prontos (gráficos de BI, tabelas de inventário).
- **Mobile (Aplicativo do Doador e Check-in da ONG):**
 - **Framework: React Native** (com Expo). Permite gerar o app para Android e iOS com um único código fonte. O Expo facilita o acesso à câmera para a leitura de **QR Code** (funcionalidade essencial do projeto).
- **Banco de Dados:**
 - **Principal: PostgreSQL** (Relacional). Essencial para garantir a consistência dos dados de inventário, auditoria, histórico de doações e usuários. Possui a extensão **PostGIS**, perfeita para a funcionalidade de geolocalização (encontrar ONGs próximas).
 - **Cache/Filas: Redis** (opcional, para gerenciar picos de acesso ou filas de notificações).

2. Definição da Arquitetura

O sistema será baseado em uma **Arquitetura de Microsserviços Pragmática** ou um **Monolito Modular** inicial, expondo uma **API RESTful**. Dado o escopo, um Monolito Modular evita a complexidade de infraestrutura inicial, mantendo o código pronto para ser separado no futuro.

- **Padrão Arquitetural do Back-end:** Baseado em **Clean Architecture (Arquitetura Limpa)** ou **DDD (Domain-Driven Design)** simplificado dentro do NestJS:
 - *Camada de Controladores (Apresentação):* Validação de rotas e entrada de dados (DTOs).
 - *Camada de Casos de Uso (Negócio):* Onde fica a lógica do "Termômetro de Urgência", regras de check-in e cálculo de impacto.
 - *Camada de Infraestrutura:* Comunicação com o PostgreSQL (via ORM como **Prisma** ou **TypeORM**), envio de e-mails/notificações e integração com mapas.
- **Padrão de Comunicação:** **API REST** estruturada, utilizando JSON para a troca de dados. As rotas serão protegidas por autenticação baseada em tokens **JWT (JSON Web Tokens)**, diferenciando os níveis de acesso (Doador, ONG, Admin).
- **Arquitetura do Front-end/Mobile:** Padrão baseado em componentes e gerenciamento de estado global unificado (ex: **Zustand** ou **Redux Toolkit**) para lidar com o estado do inventário e carrinho de doações virtuais.

3. Padrões de Código (Code Standards)

Para garantir que o código seja limpo, legível e auditável por qualquer desenvolvedor que entre no projeto social, serão adotados os seguintes padrões:

- **Estilo e Formatação:**
 - **ESLint:** Configurado com o guia de estilo do *Airbnb* ou *Google* adaptado para TypeScript, proibindo o uso de tipos **any** soltos.
 - **Prettier:** Para formatação automática de código (ponto e vírgula, aspas simples, espaçamento) no momento do salvamento do arquivo.

- **Padrões de Nomenclatura (Naming Conventions):**
 - *Variáveis e Funções:* `camelCase` (ex: `calcularTermometroUrgencia`).
 - *Classes, Interfaces e Componentes:* `PascalCase` (ex: `InventarioController`, `CardOng`).
 - *Arquivos:* `kebab-case` (ex: `ong-registration.service.ts`).
- **Mensagens de Commit (Conventional Commits):**
 - Os commits devem seguir o padrão: `tipo(escopo): descrição`.
 - Exemplos: `feat(auth): adiciona login via QR Code`, `fix(inventory): corrige cálculo do termômetro`, `chore: atualiza dependências`.
- **Testes Automatizados:**
 - **Jest** para Testes Unitários (validação do algoritmo do termômetro e regras de negócio) e Testes de Integração nas rotas críticas da API.

4. Ferramentas de Apoio

A escolha de ferramentas prioriza planos gratuitos para projetos de código aberto ou iniciativas sociais:

- **Controle de Versão e Repositório: GitHub.** Utilização de *GitHub Actions* para CI/CD (integração e deploy contínuos), garantindo que testes rodem automaticamente a cada Pull Request.
- **Gestão de Tarefas e Metodologia Ágil: Trello ou GitHub Projects** (configurado com colunas Kanban: *Backlog*, *A Fazer*, *Em Desenvolvimento*, *Revisão de Código*, *Concluído*).

- **Comunicação da Equipe:** **Discord** ou **Slack**, integrados com alertas do GitHub para avisar sobre novos commits ou erros no sistema.
- **Design e Prototipagem:** **Figma** (para o desenho das telas Web, Mobile, fluxo de check-in do QR Code e identidade visual da Rede Solidária).
- **Infraestrutura e Hospedagem (Sugestão):**
 - *Back-end e Banco de Dados:* **Supabase** (que oferece PostgreSQL e autenticação pronta em uma camada gratuita excelente) ou **Render/Railway**.
 - *Front-end Web:* **Vercel** ou **Netlify** (hospedagem gratuita e rápida para React/[Next.js](#)).

4. Ferramentas utilizadas

- Foi utilizado uso de ferramentas da la Gemini